

B1. Ingest protocol: disposition at begin, visibility by construction

C-2/C-3: the entity lock decides insert/duplicate/reproject/refuse **before** any kind row is staged; readers can only ever see the current batch.

Central observability - review **addendum**
companion to REVIEW_ADDENDUM.md - 2026-07-06

sources

writers

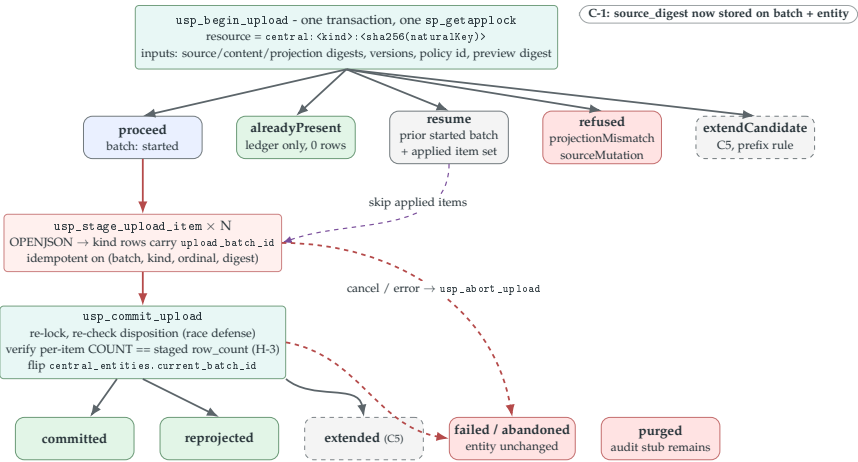
policy/contract

store

readers

CI/automation

correction/hazard



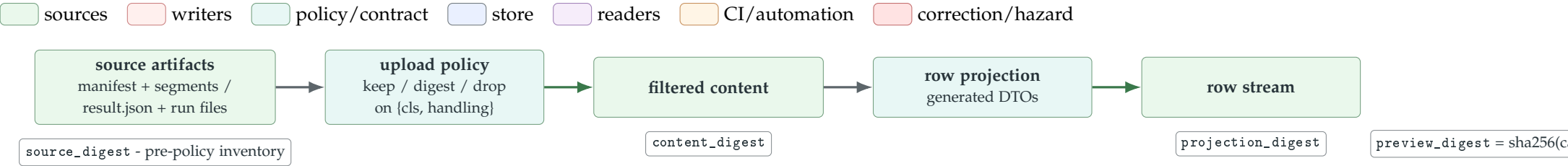
C-3 visibility rule: every canned view joins `central_entities.current_batch_id = kind.upload_batch_id`. Staged, failed, abandoned, and superseded rows are invisible by construction - no reader ever remembers a `WHERE status='committed'`. Retention gains an orphan sweep: `started > 7 d` → abandoned; non-current kind rows deleted.

refusedByPolicy = begin + abort, so refusals are ledgered too

no MERGE - applock + uniqueness constraints (base decision 4 stands)

B2. The digest algebra: why source_digest must be persisted

C-1: content_digest is policy-filtered, so a policy change alone flips it. Without the pre-policy digest, a legitimate re-upload is indistinguishable from evidence mutation.



source_digest	contract + projector + policy	content / projection	outcome	meaning
same	same	same	alreadyPresent	duplicate upload; ledger evidence, no rows
same	same	different	refused: projectionMismatch	projector nondeterminism - now an unambiguous bug signal
same	newer projector <i>or</i> different policy id	different (expected)	reprojected	legitimate; old digests preserved in ledger
different	any	any	refused: sourceMutation	source changed under an immutable key; entity untouched
prefix of incoming (same sessionId, more events)	same	extended basis	extended (C5)	growing session appended; prefix-digest proven

The failure the base design would have shipped: upload session S under team-default.v1; later re-upload the *untouched* S under team-names.v1. Same natural key, same projector, different content_digest ⇒ base §6.3 says *refused as evidence mutation*. With source_digest stored, the same event is correctly *reprojected* with the new policy recorded.

One canonicalization for everything (§6): canonicalJson moves from perftest-cli/src/run/environment.ts into perf-contracts/src/central/digest.ts. The proven environmentHash recipe ("sha256:" + hex) is unchanged and becomes the shared basis for source/content/projection/preview digests in **both** writers - one implementation, zero drift.

B3. The real DiagEvent → row mapping, and the classification ladder

C-4/C-5: the envelope is richer than the base table - required eventId, an entity anchor, tags, and a sibling GapRecord stream. cls_rank has an exact vendorable definition.

sources writers policy/contract store readers CI/automation correction/hazard

DiagEvent (sharedInterfaces/debugConsole.ts)
schemaVersion "mysql.diag.event/1"
eventId (required) - sessionId - seq
epochMs - monotonicNs?: string
process - pid? - feature - kind - type - status
traceId? - causeEventId? - **entity**?{kind,id}
durationMs? - timingClass?
payload?: Record<string, ClassifiedValue>
cls: {max, redactedFields, policyId} - tags?

central.diag_events (corrected)
session_sk int (H-1) + seq PK
event_id NOT NULL - epoch_ms
event_time_utc (projector-computed, C-4)
monotonic_ns nvarchar(40) - stays string
process/pid/feature/kind/type/status (CHECKed)
trace_id / cause_event_id
entity_kind / entity_ref (new)
duration_ms / timing_class - **tags_json** (new)
cls_max / **cls_rank** / **cls_redacted_fields**
payload_json (post-upload-policy, keeps handling)

GapRecord (journal rows) +
manifest.droppedRanges
gap_count stays as the session summary;
error-rate views flag partialWindow

central.diag_gaps (new, C-5)
gap_id - from_seq - through_seq - dropped_count
reason (subscriberOverflow / sinkOverflow / journalUnavailable)
backfill_status - first_available_seq - epoch_ms

diag_sessions.status (C-6)
exactly active | closed | partial (mirrors manifest)
"uploaded" removed - upload state lives on entity/batch
v1 precondition: only closed/partial upload;
growing sessions get the C5 extended path

provenance (C-7)
policy-filtered like payloads;
machineLabel → digest under team-default
commit / environmentHash promoted to columns

RANK_ORDER (redaction.ts) - vendor as cls-rank/1

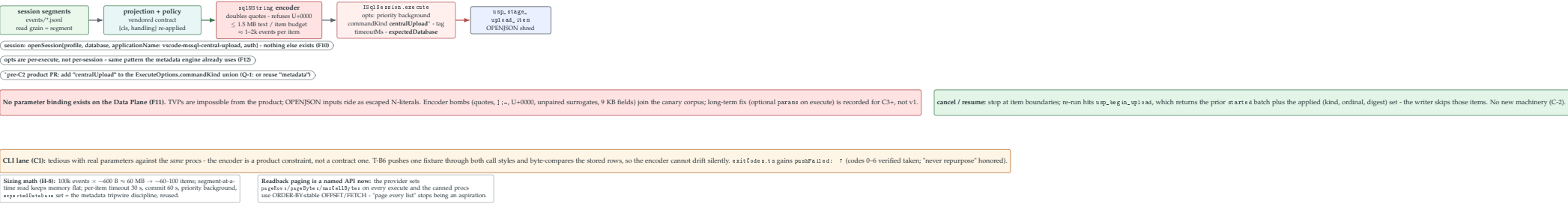
0 \cls	keep team-default
1 \cls	
2 \cls	
3 \cls	
4 \cls	
5 \cls	digest names + paths
6 \cls	
7 \cls	
8 \cls	
9 \cls	selected user.text
10 \cls	
11 \cls	drop (+ refuse: secret)
12 \cls	
13 \cls	
14 \cls	
15 \cls	
16 \cls	
17 \cls	
18 \cls	

cls_rank = index into this vendorable array; schema_info records rank_table_version; never ORDER BY cls_max - lexicographic would sort secret before sql.text. Quirk preserved: unknown outranks model classes but sits below token/connstring/secret.

B4. The product writer against the real Data Plane

C-11: `execute(text, opts, sink)` takes **no parameters**; `commandKind/priority` are per-execute. The contract owns the literal encoder, or nothing ships.

sources writers policy/contract store readers CI/automation correction/hazard



B5. The perf twin needs a subtraction list, not just additions

C-8/C-9/C-10: the local SQLite schema carries paths, machine ids, and free text a naive dialect twin would upload verbatim - and one PK that is illegal in T-SQL.

sources writers policy/contract store readers CI/automation correction/hazard

local column (perf-store.schema.sql)	class	team-default / ci	central rule
runs.output_dir - runs.config_path - repetitions.result_path	source.path	drop	absolute local paths never cross the boundary
runs.machine_id - environments.machine_id	label	digest	plain label only under team-names
runs.notes - baselines.notes	user.text	drop	keep under team-names+
artifacts.path	source.path	relative-only	projection verifies relative to run root; absolute ⇒ refusedByPolicy (Q-3)
baselines.created_by	user label	→ uploader_id FK	no free-text principals centrally
environments.*	system.metadata	keep	the join keys the feature exists for
(os/cpu/mem/versions), config_fingerprint_json, sql_image_digest	con-		
metrics.* - validations.* - repetitions	diagnostic.metadata	keep	unix-ns strings persist as truth + projector-computed *_utc for indexing
ids/status/warmup/trace/*_unix_ns			
comparisons.* - comparison_metrics.*	derived	local-only	central recomputes trends via views (Q-7)

attempt_id is real (C-9)
repetitions and metrics carry attempt_id INTEGER NOT NULL DEFAULT 0;
the central twin adds it to repetitions, metrics, validations, artifact_refs.

official_metric_samples parity surface (frozen)
join (run_id, scenario_id, rep_id, attempt_id);
WHERE m.official=1 AND r.pass_type='measurement' AND rep.status='passed'
- reproduced exactly; central extras go in official_metric_samples_ex. T-B7 byte-compares fixture rows across engines.

baselines PK
SQLite allow
Central: surr

runs.status centrally includes 'aborted' (present locally, missing from base prose) every kind table += upload_batch_id NOT NULL (C-3) goldens embody this map - conformance is byte-level (T-B5)

B6. Honest reader states, retention lanes, and purge

C-12 + H-1/H-4: the state ladder is a real union extension in perfHistory.ts; filtered-ness is a query-result fact; retention deletes whole sessions.

sources writers policy/contract store readers CI/automation correction/hazard

PerfSourceStatus - existing union (perfHistory.ts)

indexed scanning partial stale error empty unsupported unavailable permissionDenied ← C-12 additions; PerfSourceKind += "central"

"Filtered" is not a source state. Central query responses gain resultFlags?: ("officialOnly" | "policyDropped" | "partialWindow" | "rowCapped")[] - a list can be indexed and visibly filtered at once. This is how the base's five-way empty/filtered/failed/dropped/denied distinction actually reaches the UI. The sqlite source's status:"unsupported" registration is the copy-pattern for "configured but data plane cannot open": register visible, never hide.

provider rules (C3)
every execute sets pageRows / pageBytes / maxCellBytes;
reads only views + iTVFs (role-denied on base tables);
trend parity vs direct SQL is a gate (T-B12);
source appears only when configured.

diag sessions - whole-session TTL

keyed by session_sk (H-1): delete gaps, events,
session row in one sweep; 90 d default;
no row-window deletes over a wide key

detail tiers

markers 30 d - sql_activity 30 d
batched TOP-n loops;
SQL-Agent-free (base rule kept)

orphan sweep (C-3)

started > 7 d → abandoned;
kind rows not referenced by any
current_batch_id deleted after 7 d

usp_purge_entity effects (base §10.3, made checkable)

kind rows + gaps deleted - entity.purged_at stamped - batch rows keep counts only
(source_summary scrubbed of labels) - uploader display cleared on privacy purge
- ledger audit stub remains - T-B13 asserts every line

central_health additions (H-6)

schema/contract/rank-table/union versions vs expected - last cleanup
- failed/refused batches 7 d - largest sessions - orphaned staged rows
- canary projection check row - CI heartbeat (receipt from last C1' publish)

views distinguish: no data / filtered / upload failed / policy dropped / permission denied - never a lying "empty"

min platform pinned: SQL 2019+/Azure SQL, compat ≥ 150 (H-2)